

MINI ASSESSMENT

Robust Handling of API Rate Limits and Malformed Responses

Name: Vummadichetty Jaya Varshini

Student ID: 99230040812

Date: 11-09-26

Q1. If an API returns 429, what should your program do?

A 429 response tells the client it has sent too many requests within the allowed window — it is a throttling signal, not an indication that anything is actually broken. Because the underlying request was perfectly valid, the right response is patience rather than panic: the program should pause and try again shortly instead of surfacing an error to the user or abandoning the operation outright.

The standard technique is exponential backoff — wait briefly, retry, and if the limit is hit again, lengthen the wait each time (typically doubling it) until either the call succeeds or a retry ceiling is reached. Well-behaved APIs usually include a `Retry-After` header telling the client precisely how long to pause; a robust client reads that value instead of picking an arbitrary delay.

```
import time
import requests

def fetch_with_retry(url, retries=5):
    wait_time = 1
    for attempt in range(retries):
        resp = requests.get(url)
        if resp.status_code != 429:
            return resp
        pause = int(resp.headers.get('Retry-After', wait_time))
        time.sleep(pause)
        wait_time *= 2
    raise RuntimeError('Rate limit persisted after all retries')
```

Skipping this logic and simply looping until a request succeeds is a common mistake — it floods the provider with doomed requests and can escalate to the client's API key being throttled even harder or suspended entirely. Backoff turns a rate limit into a brief, self-resolving delay instead of a program crash.

Q2. If `response.json()` fails, what should your program do?

Calling `.json()` on a response assumes the body is well-formed JSON, which is not guaranteed — the server might instead return an HTML error page, an empty payload, plain-text output, or a response cut short by a dropped connection. When that assumption is wrong, `.json()` throws a decoding error, and letting that exception propagate unhandled will bring the whole program down over what is often a recoverable, transient issue.

A safer approach checks the response's status code and content type before parsing, since a failing status or an unexpected content type is a strong hint that JSON parsing is going to fail anyway. The parse call itself belongs inside a try/except so a bad response is captured, logged with enough context to diagnose later, and replaced with a controlled fallback rather than an unhandled crash.

```
resp = requests.get(url)

try:
    payload = resp.json()
except ValueError as e:
    print(f'Could not parse JSON (HTTP {resp.status_code}): {e}')
    print('Raw body preview:', resp.text[:200])
    payload = {}
```

Keeping a snippet of the raw response text in the log is worth the extra line — it is frequently the fastest way to spot the real cause, such as an upstream service quietly returning a maintenance page instead of the expected data.

Q3. If a JSON key is missing, why might `.get()` be safer than `[]`?

Square-bracket lookup on a dictionary, like `payload['status']`, demands that the key exist — if it doesn't, Python raises a `KeyError` and the program stops right there unless that error is specifically caught. Real API payloads are rarely that rigid: optional fields get omitted, fields appear only under certain conditions, and schemas evolve over time without every client being updated in lockstep.

The `.get()` method sidesteps this fragility. `payload.get('status')` simply returns `None` when the key is absent instead of raising an exception, and a second argument lets the caller supply a meaningful default, as in `payload.get('status', 'unknown')`. Either way, the program keeps executing and the missing-field case becomes a deliberate branch in the logic rather than an unhandled failure.

```
record = {'id': 101, 'name': 'Anita Rao'}

# KeyError if 'status' isn't present:
status = record['status']

# None instead, no crash:
status = record.get('status')

# A usable fallback instead of None:
status = record.get('status', 'unknown')
```

This matters most when a program is iterating over many API records in a batch — one record missing an optional field shouldn't be able to halt the entire job. `.get()` with sensible defaults keeps the pipeline resilient to small schema drift, while `[]` access is best reserved for fields whose presence is truly guaranteed, where a missing key really does signal a bug worth failing loudly for.